Backend Docker Deployment Cheat Sheet

A plain-English AWS Lightsail runbook organized by development change type

Purpose

Use this guide after reviewed development changes have been merged and pushed to **main**. Start with the decision table, run only the actions needed for that release, and always finish with verification.

> Golden rule: Pull first. Rebuild when application code or dependencies changed. Migrate when models changed. Collect static files when Django admin/static assets changed. Then recreate the backend and verify it.

Production services

Service	What it does
web	Django + Gunicorn; container **tox-backend-prod**
postgres	PostgreSQL; container **tox-postgres-prod**
Nginx / Cloudflare	Routes public HTTPS traffic to 127.0.0.1:8000
S3	Stores production media when `USE_S3=true`

> Stop if `git status` shows uncommitted server changes. Never expose backend/.env values. Never run `docker compose down -v` in production.

1. Choose commands based on what changed

If a release contains several change types, combine their required actions.

Development change	Actions required on Lightsail
Python code: views, serializers, admin, services	Pull; build web; start updated web; verify
requirements.txt	Pull; build web; start updated web; verify imports and logs
Models or migration files	Pull; build web; start postgres; migrate; start web; verify
Admin CSS or static files	Pull; build web; collectstatic; start web; verify admin
backend/.env values	Edit env securely; force-recreate web; verify configuration
Dockerfile or production Compose	Pull; review diff; rebuild; recreate affected services; verify
Frontend React only	No backend deployment; Vercel handles frontend after push
Data fix / management command	Back up DB; run only the reviewed command; verify data

Current series workflow release CODE ONLY

Backend Python admin code and tests changed. No model field or migration changed. Rebuild/restart web. Running `migrate` remains safe and confirms the schema is current.

> Confirm the release commit exists on origin/main before using Lightsail. A local merge cannot be pulled until it is pushed.

2. Safe default deployment order

#	Command	What it does
1	`git status -sb`	Checks branch and detects server changes.
2	`git switch main`	Selects the production branch.
3	`git pull --ff-only origin main`	Downloads reviewed code; refuses an accidental merge.
4	`docker compose --env-file backend/.env -f docker-compose.prod.yml build web`	Builds a new Django image. Does not start it.
5	`docker compose --env-file backend/.env -f docker-compose.prod.yml up -d postgres`	Ensures PostgreSQL runs in the background.
6	`docker compose --env-file backend/.env -f docker-compose.prod.yml run --rm web python manage.py migrate`	Runs migrations in a temporary web container.
7	`docker compose --env-file backend/.env -f docker-compose.prod.yml run --rm web python manage.py collectstatic --noinput`	Copies admin/static assets when needed.
8	`docker compose --env-file backend/.env -f docker-compose.prod.yml up -d web`	Creates/replaces web with the new image.
9	`docker compose --env-file backend/.env -f docker-compose.prod.yml ps`	Displays status and database health.

Routine shortcut

This builds and starts web in one command. Use the longer sequence when migrations, static collection, or troubleshooting require explicit control.

3. Docker Compose command dictionary

Command part	Plain-English meaning
`--env-file backend/.env`	Loads production substitutions used by Compose, including PostgreSQL values. Keep it on production commands; never print the file.
`-f docker-compose.prod.yml`	Selects production configuration, not local development.
`config --quiet`	Validates Compose and environment substitutions without printing resolved values.
`build web`	Creates a new image. The running container is unchanged.
`build --no-cache web`	Rebuilds every layer. Slower; use for suspected stale cache.
`up -d web`	Starts/recreates web. `-d` means detached.
`up -d --build web`	Builds and starts web in one step.
`up -d --force-recreate web`	Recreates web even when the image is unchanged; useful after environment edits.
`run --rm web COMMAND`	Runs a temporary container, then removes it.
`exec web COMMAND`	Runs inside the already-running web container.
`ps`	Shows service status, ports, and health.
`logs --tail=100 web`	Shows the latest 100 backend log lines.
`logs -f web`	Follows live logs; Ctrl+C only stops viewing.
`restart web`	Restarts the existing container; does not rebuild code.
`down`	Stops/removes containers and network. Avoid routinely.
`down -v`	**DANGER:** removes volumes and can delete database data.

> Critical distinction: `restart web` is not a code deployment. New source code requires `build web` followed by `up -d web`.

4. Ready-to-run recipes

Normal backend code update

Model and migration update

Admin CSS or static asset update

Environment update

Dependency update

5. Verify and recover

Verification commands

1. Both production containers should be Up; PostgreSQL should be healthy.
1. Logs should have no import errors, database failures, or repeated worker crashes.
1. The admin check should return 200 or a redirect such as 301/302.
1. Test the exact workflow changed by the release.

Troubleshooting order

1. Read logs: `docker compose --env-file backend/.env -f docker-compose.prod.yml logs --tail=200 web`.
2. Confirm deployed commit: `git log -1 --oneline`.
3. Confirm required env variables exist without printing secret values.
4. Never use `down -v` or reverse migrations without a reviewed plan.

Code rollback

Current release check

Item	Expected
Main merge	`b2243eb`
Migration / static collection	Not required
Functional test	Create series draft -> season draft -> episode source -> submit/approve

6. Pre-deployment gate and database safety

Connect, enter the repository, and inspect

Gate	Expected result
Git state	Branch is `main`; there are no unexplained server-side edits.
Remote release	The intended commit exists on `origin/main`. Lightsail cannot pull a commit that only exists locally.
Compose validation	`config --quiet` returns no error and does not expose resolved values.
Service baseline	Web is Up; PostgreSQL is Up and healthy before deployment begins.

Create a timestamped PostgreSQL backup

`-T` disables pseudo-terminal output so redirected SQL is clean. Database variables expand inside the container; their values are not printed by the command itself. Replace `<TIMESTAMP>` with a date-time label.

Minimum backup validation

> Never commit a backup to Git. A backup stored only on the same Lightsail instance is not disaster recovery; copy approved backups to protected storage under your retention policy.

Migration rules

1. Deploy committed, reviewed migration files only.
1. Never run `makemigrations` on production.
1. Back up before schema or production data changes.
1. Do not reverse a migration without reviewing data impact.

7. Environment, Compose, Nginx, and disk commands

Environment-only update

An env change normally needs container recreation, not an image build. Build as well if code, requirements, or the Dockerfile changed.

Production Compose update

Review service names, bind mounts, named volumes, ports, and PostgreSQL settings before applying a Compose change.

Nginx-only update

Reload only after `nginx -t` succeeds. Nginx-only changes do not require a Docker rebuild.

Run a reviewed Django command

Inspect disk usage and reclaim dangling images

Command	Production meaning
`image prune -f`	Removes dangling images only. Run after the new release is verified.
`system prune -a`	Broad cleanup; can remove useful cached and unused images. Do not use routinely.
`volume prune`	Can remove unused volumes. Avoid unless every candidate volume is reviewed.

8. Verify from container to public HTTPS

Container and commit checks

Local Gunicorn and public endpoint checks

The explicit Host header matters because Django production allowed-host rules may reject a direct `127.0.0.1` request. A `200`, `301`, or `302` is normally acceptable for the admin path.

What clean verification looks like

1. Both services are `Up`; PostgreSQL reports `healthy`.
1. Gunicorn starts workers without repeated exits.
1. No import, database, S3, allowed-host, or missing-env errors appear.
1. The checked-out SHA equals the intended release.
1. Local and public HTTP checks return valid status codes.
1. The exact workflow changed in development works in production.

Diagnostic commands, in order

Follow the boundary: container failure → Docker/web logs; local HTTP works but public fails → Nginx/Cloudflare/TLS; database unhealthy → PostgreSQL logs and disk state.

9. Lightsail deployment checklist

Before

1. Release is merged into `main` and pushed to `origin/main`.
1. Release type is identified from page 2.
1. A database backup exists when migrations or data changes are involved.
1. The server working tree has no unexplained edits.

Deploy

After

> Deployment is complete only when services are healthy, logs are clean, the SHA matches, HTTP checks respond correctly, and the changed workflow passes.

Current release acceptance check

Release	Manual production check
`b2243eb`	Create a series draft → create a season draft before an episode exists → create the episode/source → complete the intended submit/approve flow.

> **Never use routinely:** `down -v`, `volume prune`, production `makemigrations`, force-push, or an unreviewed database restore.

Prepared from this repository's production Compose file, backend Dockerfile, and Lightsail deployment notes. Updated 28 June 2026.